13. Implement your own ping program.

Experiment No. 13

Experiment Name

Implementation of Own Ping Program

Aim

To develop a ping program using Python to check the connectivity between a source computer and a destination host. The program sends ICMP Echo Request packets and receives ICMP Echo Reply packets to determine whether the destination is reachable and to calculate the round-trip time.

Procedure

1. **Open Visual Studio Code and create a folder named CN_Experiment_13. Create a Python file named ping_program.py inside the folder.**
2. **Ensure that Python is installed on the system. Open the VS Code terminal and execute python --version to verify the installation.**
3. **Import the required modules, including socket, struct, time, and select. These modules provide socket communication, packet-header construction, time measurement, and waiting for incoming responses.**
4. **Create an ICMP Echo Request packet. The packet contains an ICMP type field, code field, checksum, identifier, sequence number, and optional payload data. For an ICMP Echo Request, the type is 8 and the code is 0 for IPv4.**
5. **Implement a checksum calculation function to calculate the checksum required for the ICMP packet. The checksum helps detect errors in the packet data.**
6. **Create a RAW socket using the IPv4 address family, RAW socket type, and ICMP protocol. The socket allows the program to send ICMP packets and receive ICMP responses.**
7. **Ask the user to enter the hostname or IP address of the destination computer. Resolve a hostname into an IP address if necessary, using the appropriate socket function.**

8. **Construct the ICMP Echo Request packet with the calculated checksum, identifier, and sequence number. Record the start time immediately before sending the packet to the destination.**
9. **Send the packet using the socket's sendto() function. The destination IP address is specified as the target of the packet transmission.**
10. **Wait for an incoming packet using a timeout. If a response is received, examine its IP and ICMP headers to identify the ICMP Echo Reply corresponding to the request. Record the time at which the response arrives.**
11. **Calculate the round-trip time by subtracting the sending time from the receiving time. Display the destination IP address, packet sequence number, and measured response time.**
12. **If no valid response arrives before the timeout expires, display a message indicating that the request timed out. A timeout may occur because the destination is unreachable, ICMP traffic is blocked, or the network is experiencing packet loss.**
13. **Repeat the process for several packets by increasing the sequence number for each request. Record the number of packets sent, replies received, lost packets, and response times.**
14. **Run the program from the VS Code terminal. On Windows, RAW ICMP sockets may require administrator privileges. If the operating system denies access to the socket, run the terminal with the required privileges or use an operating-system-supported ICMP method.**
15. **Compare the output of the Python program with the built-in ping command. For example, execute ping 127.0.0.1 to test the local loopback interface. Observe the replies and response times, and note any differences between the two outputs.**

ping_program.py

```
import socket

import time
```

BALAGANESH P- CN-13

```
host = input("Enter host/IP address: ")


try:
    ip = socket.gethostbyname(host)
    print("Pinging", host, "[", ip, "]")

    for i in range(4):
        start = time.time()

        try:
            s = socket.create_connection((ip, 80), timeout=2)
            end = time.time()

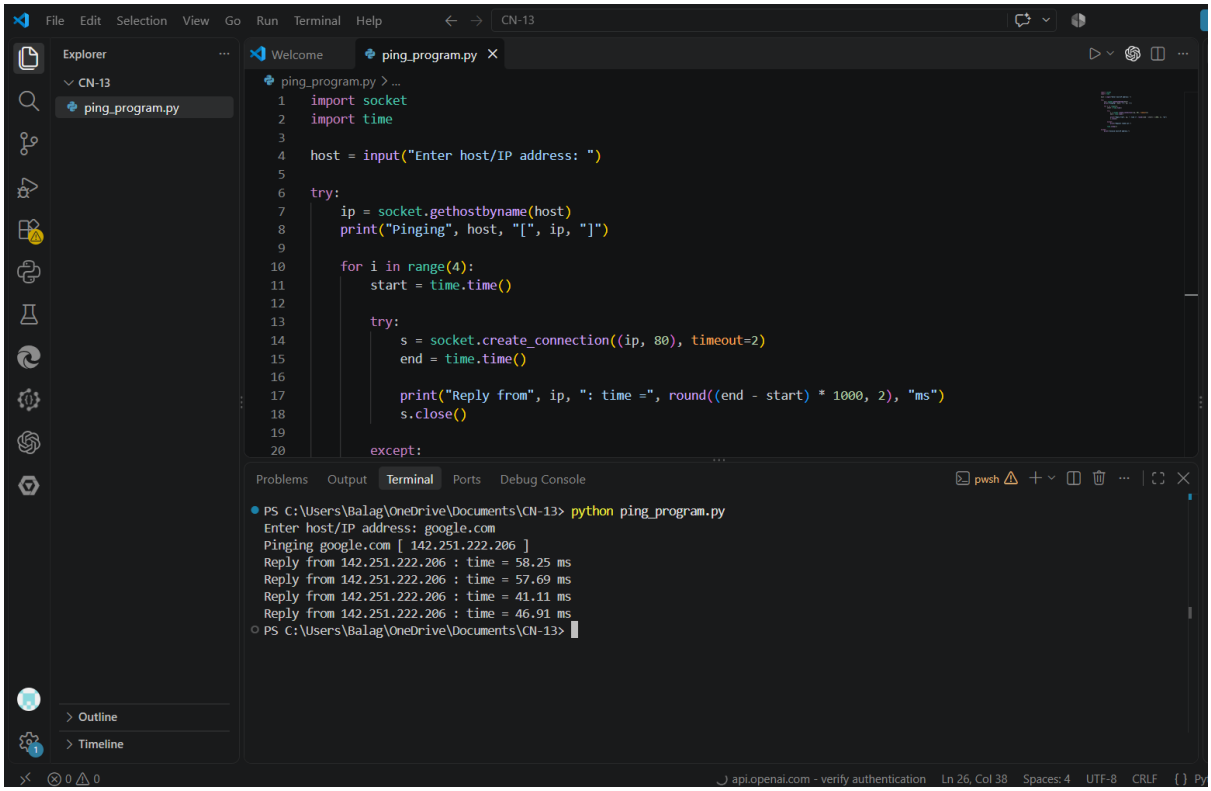
            print("Reply from", ip, ": time =", round((end - start) * 1000, 2), "ms")
            s.close()

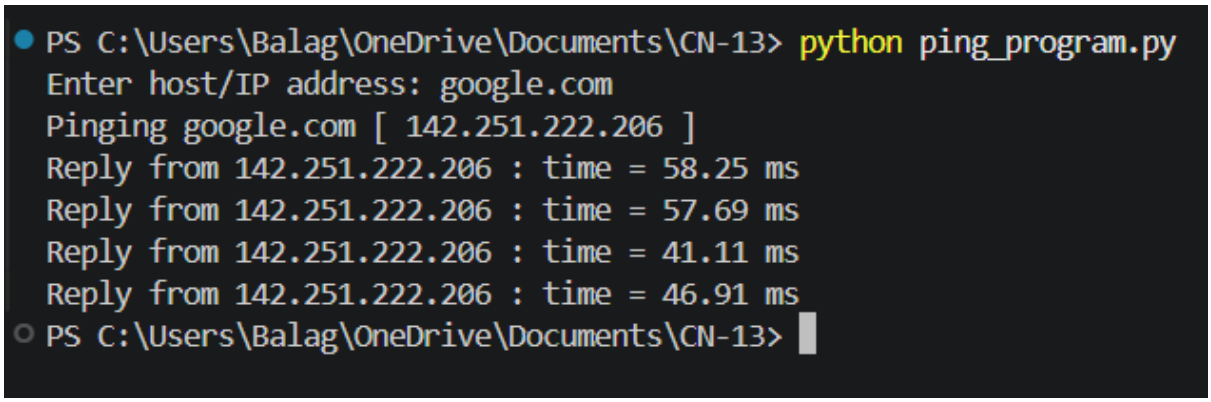
        except:
            print("Request timed out.")

        time.sleep(1)

except:
    print("Invalid host/IP address.")
```

BALAGANESH P- CN-13





Result

Thus, an ICMP-based ping program was designed to test network connectivity between a source computer and a destination host. The program sends ICMP Echo Request packets, processes the corresponding Echo Reply packets, measures round-trip time, and identifies requests that time out. The experiment demonstrates the use of ICMP for network diagnostics and connectivity testing.